

Project Learning Guide

Real-Time OV7670 Video Capture and VGA Processing System

Basys 3 / Artix-7 Verilog Code Walkthrough

May 2026

Abstract

This guide explains what is happening in this project and how the Verilog works. It assumes you have some Verilog background, such as a Basys 3 calculator project, but may have forgotten details about FPGA clocks, camera buses, memories, and VGA. It only explains hardware and project-description details that are needed to understand this codebase.

Contents

1 Project Goal	2
2 Repository Map	2
3 Hardware Background	2
3.1 Basys 3 FPGA Board	2
3.2 OV7670 Camera	3
3.3 VGA Output	3
3.4 Block RAM	3
4 Clocks, Reset, and CDC	4
5 Camera Configuration Path	4
6 Camera Capture Path	5
7 Display Path	6
8 Filter Engine	7
9 Top-Level Integration	7
10 Constraints and Physical Pins	8
11 Simulation and Implementation Evidence	8
12 Debugging Mental Model	9
13 Main Takeaways	9

1 Project Goal

The project makes a Basys 3 FPGA act as a live video processor. The OV7670 camera sends pixels to the FPGA. The FPGA stores a downsampled frame in block RAM, reads it for VGA output, applies a switch-selected image filter, and drives a monitor.

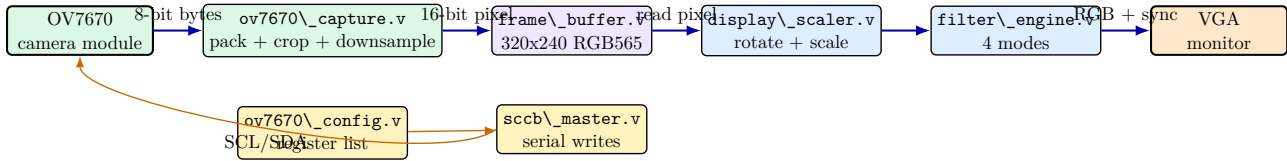


Figure 1: High-level data path and separate camera-configuration path.

2 Repository Map

Path	What it contributes
hdl\sources\top.v	Top-level integration: clocks, reset, SCCB, capture, BRAM, VGA, display, CDC.
hdl\sources\ov7670_capture.v	Converts camera bytes into RGB565 writes.
hdl\sources\ov7670_config.v	69 camera register writes and 2 ms delay sequencing.
hdl\sources\sccb_master.v	Generates the OV7670 SCCB serial write waveform.
hdl\sources\frame_buffer.v	Wraps the Xilinx true dual-port block RAM IP.
hdl\sources\display_scaler.v	Maps VGA coordinates to frame-buffer addresses and gates display output.
hdl\sources\filter_engine.v	Raw, grayscale, red-only, and inverted RGB565 filters.
hdl\sources\vga_controller.v	640x480 VGA timing counters and sync pulses.
hdl\sources\constraints.xdc	Basys 3 pin assignments and clock constraints.
sim/test*.py	Cocotb tests for the key modules.
ov7670\vga_system.runs\impl_1	Vivado implementation reports and bitstream.

Table 1: The files that matter for understanding the working design.

3 Hardware Background

3.1 Basys 3 FPGA Board

The Basys 3 board provides the Artix-7 FPGA, a 100 MHz oscillator, switches, buttons, LEDs, Pmod pins for the camera, and VGA output pins. In a simple calculator project, you may have used one clock and a few switches. This project uses the same Verilog ideas, but with external timing devices: a camera and a monitor.

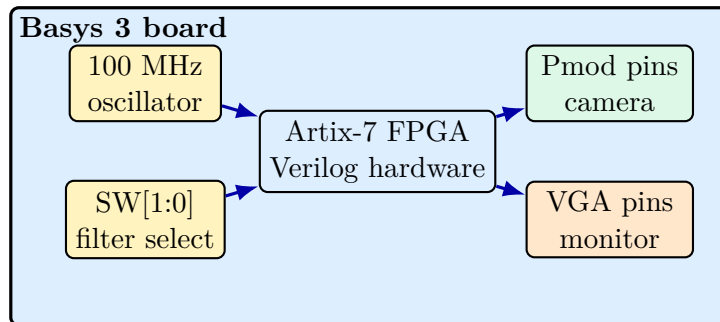


Figure 2: The FPGA is the hardware you design. The board supplies clock, switches, connectors, and VGA DAC wiring.

3.2 OV7670 Camera

The OV7670 sends one byte at a time on `cam\_d[7:0]`. Two bytes make one RGB565 pixel. The camera also provides timing signals:

Signal	Direction	Meaning in this project
<code>cam_d[7:0]</code>	camera to FPGA	One pixel byte at a time.
<code>cam_pclk</code>	camera to FPGA	Byte clock. The capture module samples on negative edge.
<code>cam_href</code>	camera to FPGA	High while a valid row is being transmitted.
<code>cam_vsync</code>	camera to FPGA	High during frame reset/blanking; falling edge means a frame finished.
<code>cam_xclk</code>	FPGA to camera	Master clock supplied by the FPGA through an ODDR output.
<code>cam_scl, cam_sda</code>	FPGA to camera	SCCB register-write bus used during startup.

Table 2: Only these camera concepts are needed to understand the code.

16-bit RGB565 pixel stored in BRAM

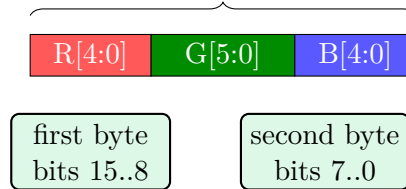


Figure 3: `ov7670\_capture.v` combines two 8-bit transfers into one 16-bit RGB565 pixel.

3.3 VGA Output

VGA is a timing standard. For 640x480 output the design counts 800 pixel clocks per line and 525 lines per frame. The visible part is only 640x480; the rest is blanking and sync timing.

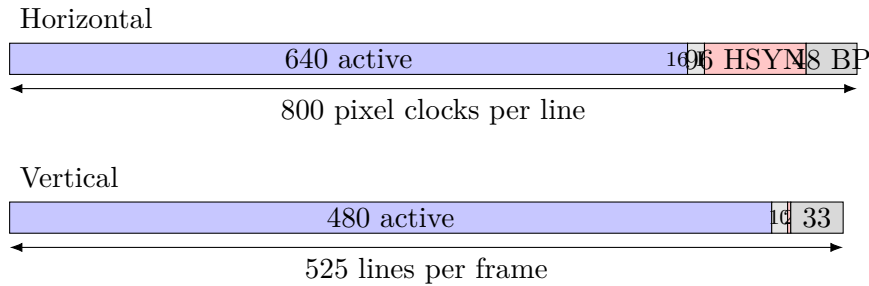


Figure 4: `vga\_controller.v` implements these standard 640x480 timing ranges.

3.4 Block RAM

The Basys 3 has enough block RAM for a 320x240 RGB565 frame, but not a full 640x480 RGB565 frame:

$$320 \times 240 \times 16 = 1,228,800 \text{ bits}$$

The code uses a true dual-port BRAM. The camera writes on `cam\_pclk`; the VGA reads on `pixel\_clk`. This is the main boundary between unrelated clocks.

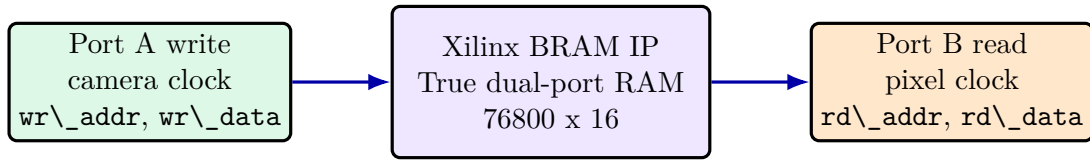


Figure 5: `frame\_buffer.v` wraps this BRAM IP and provides a simulation model under `SIMULATION`.

4 Clocks, Reset, and CDC

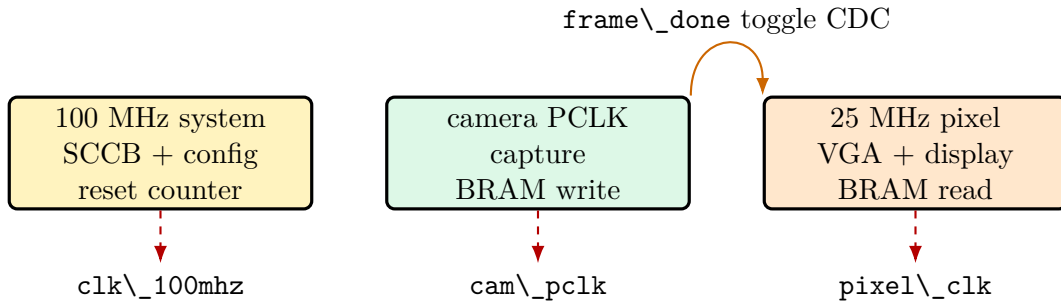


Figure 6: The design is not single-clock. Crossings must be handled deliberately.

`top.v` waits for the Clocking Wizard lock signal, then holds reset for a 24-bit counter:

$$2^{24}/100,000,000 \approx 0.168 \text{ seconds}$$

Then it synchronizes reset into the camera and pixel domains with two flip-flops. For frame completion, it uses a pulse-to-toggle synchronizer so a one-cycle camera-domain pulse is not missed by the pixel clock.

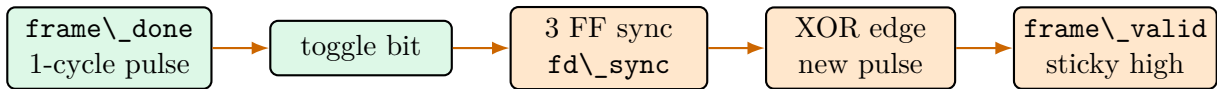


Figure 7: The display stays black until a complete camera frame has been captured.

Clock note. The top comments mention a 24 MHz camera clock in one place, but the generated `clk\_wiz\_0.xci` currently requests 25 MHz for output 1 and output 2. The design idea is unchanged: output 1 is the VGA pixel clock, and output 2 is sent to the camera through an ODDR. If exact 24 MHz is required, edit Clocking Wizard output 2.

5 Camera Configuration Path

`ov7670\_config.v` contains 69 register writes. The full datasheet is not needed to follow the project. What matters is that the writes set reset, clocking, RGB565 format, windowing, exposure/gain/white balance, gamma, color matrix, and stability options.

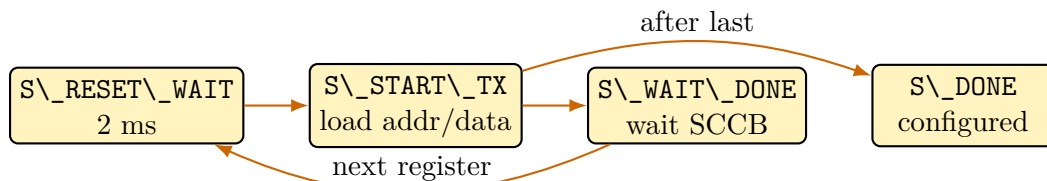


Figure 8: The config sequencer waits between every SCCB write because the camera needs settling time.

`sccb\_master.v` sends one write transaction as START, device address, register address, register data, and STOP. With `dev\_addr = 7'h21`, the ID byte is `8'h42`.

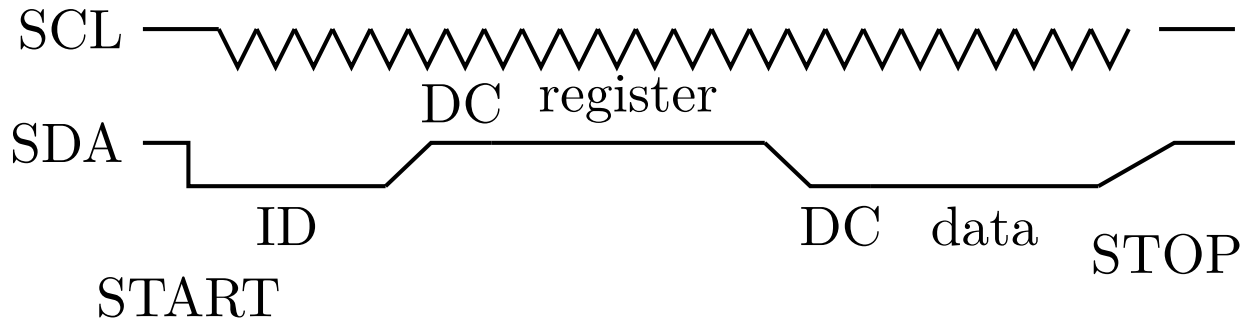


Figure 9: Simplified SCCB write waveform. The RTL generates the 100 kHz SCL from the 100 MHz clock.

6 Camera Capture Path

`ov7670\_capture.v` runs on `negedge pclk`. It stores the first byte, then combines it with the second byte to form one RGB565 pixel. It also skips unstable edge data and keeps only even row/even column positions.

```
if (h_cnt >= 10'd4 && v_cnt >= 10'd4 &&
    h_cnt < 10'd644 && v_cnt < 10'd484) begin
    if (h_cnt[0] == 1'b0 && v_cnt[0] == 1'b0) begin
        wr_data <= {first_byte, cam_d};
        wr_en    <= 1'b1;
        wr_addr <= ((v_cnt >> 1) - 2) * 320 + ((h_cnt >> 1) - 2);
    end
end
```

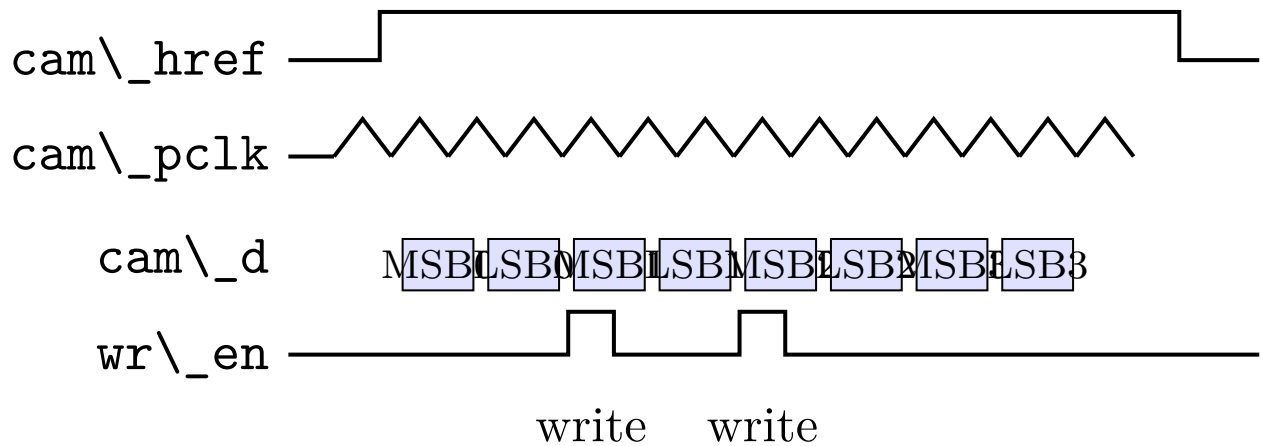


Figure 10: Two camera bytes become one BRAM write. `wr\_en` is a one-cycle write pulse.

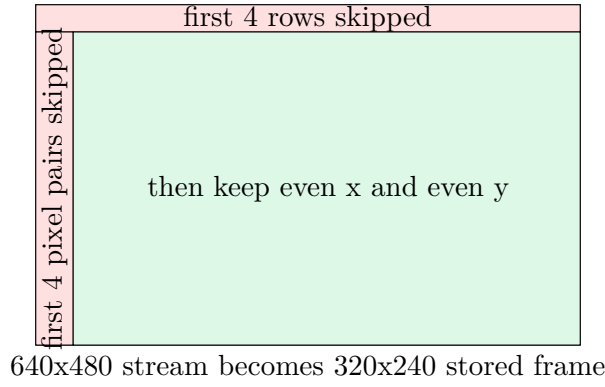


Figure 11: The crop removes unstable camera edges; the even/even rule downsamples the frame.

The frame-buffer address is:

$$\text{addr} = y \times 320 + x = (y \ll 8) + (y \ll 6) + x$$

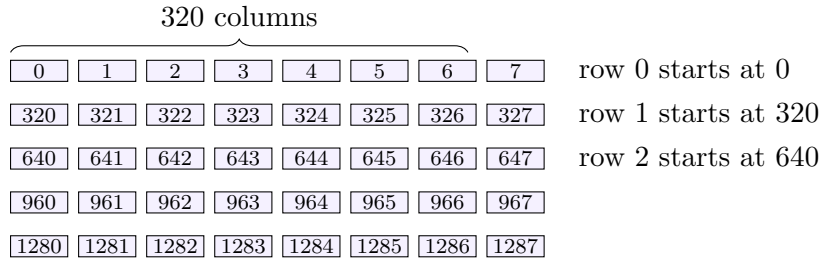


Figure 12: A 2D image is stored in 1D memory by row-major addressing.

7 Display Path

`vga\_controller.v` produces `hcount`, `vcount`, active-low HSYNC, active-low VSYNC, and `active`. `display\_scaler.v` uses the counters to choose a BRAM address.

The implemented display mapper centers a 360x480 window on the 640x480 screen, scales it down using 171/256, then swaps coordinates to rotate/mirror the stored image:

```
valid_area = hcount >= 140 && hcount < 500 && vcount < 480;
screen_x   = hcount - 140;
rot_x      = (screen_x * 171) >> 8; // 0..239
rot_y      = (vcount * 171) >> 8; // 0..319
img_x      = rot_y;
img_y      = rot_x;
addr       = img_y * 320 + img_x;
```

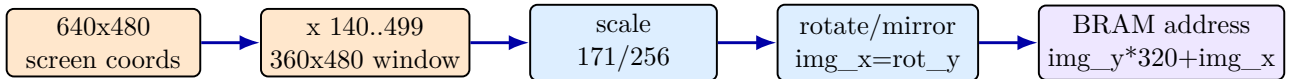


Figure 13: Display coordinate mapping used by `display\_scaler.v`.

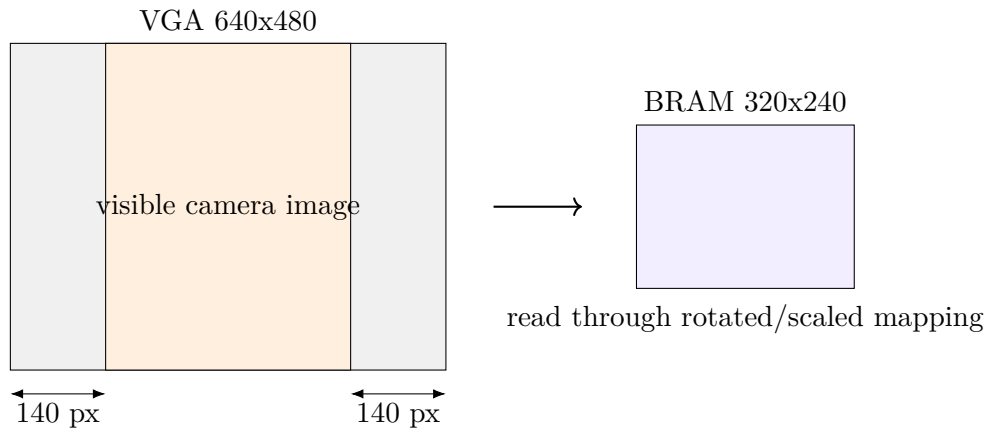


Figure 14: The display image is centered and orientation-corrected.

8 Filter Engine

The filter engine is combinational. It takes one RGB565 input pixel and returns one RGB565 output pixel.

SW1	SW0	Mode	Operation
0	0	Raw	Copy the pixel unchanged.
0	1	Grayscale	Compute brightness from R/G/B and copy it into all channels.
1	0	Red only	Keep red bits, zero green and blue.
1	1	Invert	Bitwise invert all 16 RGB565 bits.

Table 3: Switch-selected modes in `filter\_engine.v`.

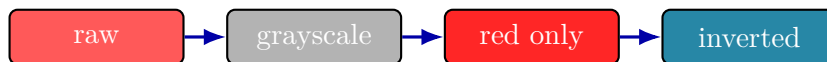


Figure 15: The filter changes each pixel independently; there is no neighborhood or line-buffer processing.

The grayscale calculation uses integer arithmetic:

$$y_scaled = R5 \times 54 + G6 \times 183 + B5 \times 18$$

$$y_6bit = y_scaled[13 : 8], \quad y_5bit = y_6bit[5 : 1]$$

9 Top-Level Integration

`top.v` is the wiring point for the whole system. Its main jobs are:

- instantiate `clk\_wiz\_0` and ODDR for the outgoing camera clock;
- hold camera PWDN low and camera reset high;
- run SCCB config on the 100 MHz domain;
- capture camera pixels in `cam\_pclk`;
- write/read the dual-port BRAM;
- produce VGA counters and sync;

- Basys 3 Artix-7 design with camera capture, dual-port frame buffering, image filtering, and 640x480 VGA output

Testbench	What it checks
<code>test_filter_engine.py</code>	All four filter modes and switch transitions.
<code>test_vga_controller.py</code>	Reset, counter wrap, active area, HSYNC, VSYNC.
<code>test_ov7670_capture.py</code>	Frame done pulse, edge guard, first writes, write-enable pulse.
<code>test_sccb_master.py</code>	Idle state, busy/done, SCL toggling, START, back-to-back transactions.

Table 5: Module-level simulation gives confidence before programming the FPGA.

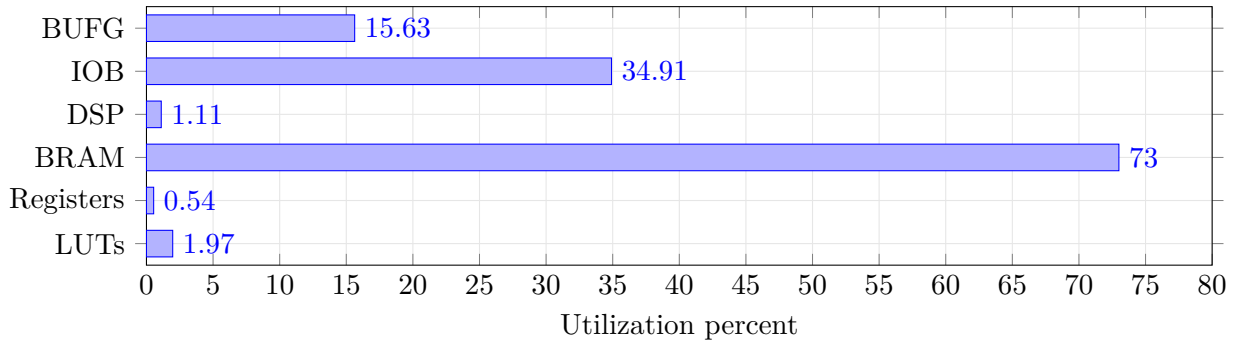


Figure 17: Vivado utilization: the logic is small, but the frame buffer consumes most BRAM.

Report item	Value
LUTs	409 of 20,800, or 1.97 percent
Registers	224 of 41,600, or 0.54 percent
BRAM tiles	36.5 of 50, or 73.00 percent
Worst setup slack in timing summary	13.021 ns
Worst hold slack in timing summary	0.154 ns
Estimated total on-chip power	0.197 W, low confidence

Table 6: Implementation summary from routed Vivado reports.

12 Debugging Mental Model

For hardware debug, follow the data path from left to right. Later blocks cannot work if earlier blocks are dead.

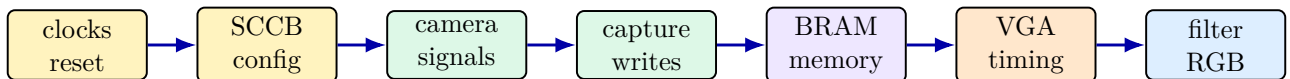


Figure 18: Recommended debug order for this project.

13 Main Takeaways

- The camera sends bytes. The FPGA creates RGB565 pixels from byte pairs.
- The frame buffer is the heart of the design because it bridges camera and VGA timing.
- 320x240 RGB565 fits in Basys 3 BRAM; full 640x480 RGB565 does not.
- VGA output is mostly counters plus active-low sync pulses.
- Clock-domain crossings are handled with BRAM for pixels and a toggle synchronizer for frame completion.

- `top.v` is the best file to start reading because it shows how every hardware block connects.